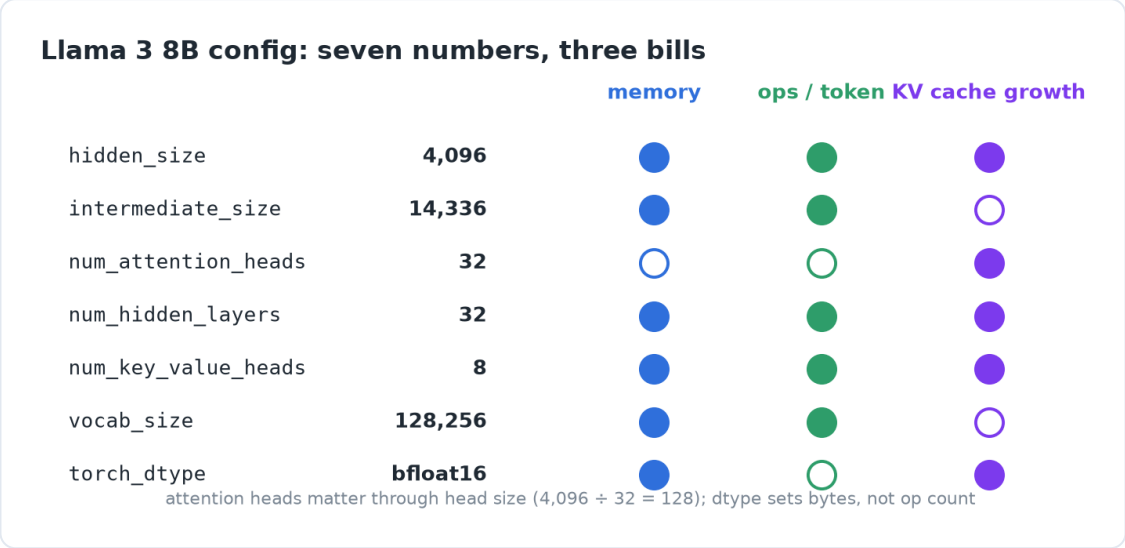
07 · Transformer Architecture Refresher

Source: fanout.sh / inference-eng / 00-06 · ~9 min

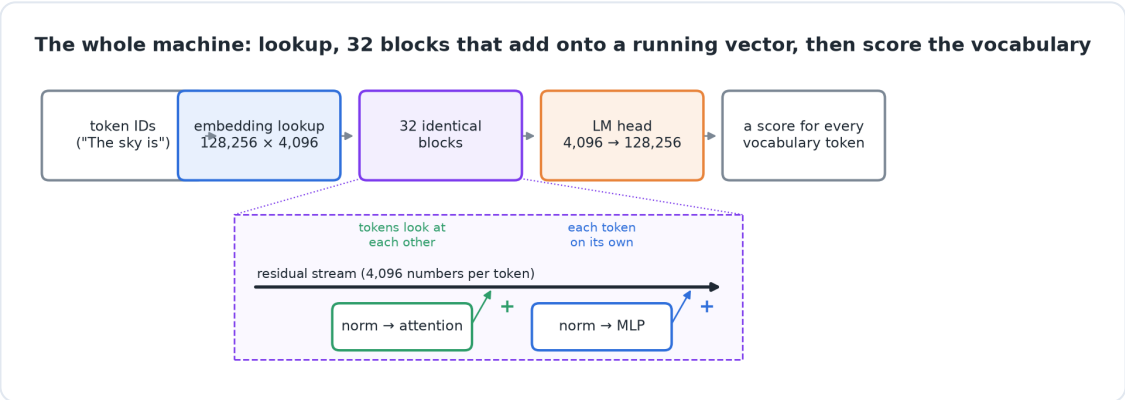
IN SHORT

A transformer is an **embedding lookup**, a stack of **identical blocks** (attention + MLP, each adding onto a running vector), and a **head** that scores the whole vocabulary. Seven numbers in Llama 3 8B's ~30-line config decide its whole bill: **8.03B parameters** (70% in the MLPs), **~15B operations per token**, and a KV cache of **128 KB per token**. New models mostly swap parts of this skeleton, and each swap moves one line of the bill.



The seven config numbers that matter, and which bill each one feeds.

1. The whole machine



Token IDs pick rows of the embedding table; 32 blocks add their results onto the residual stream; the LM head scores all 128,256 tokens.

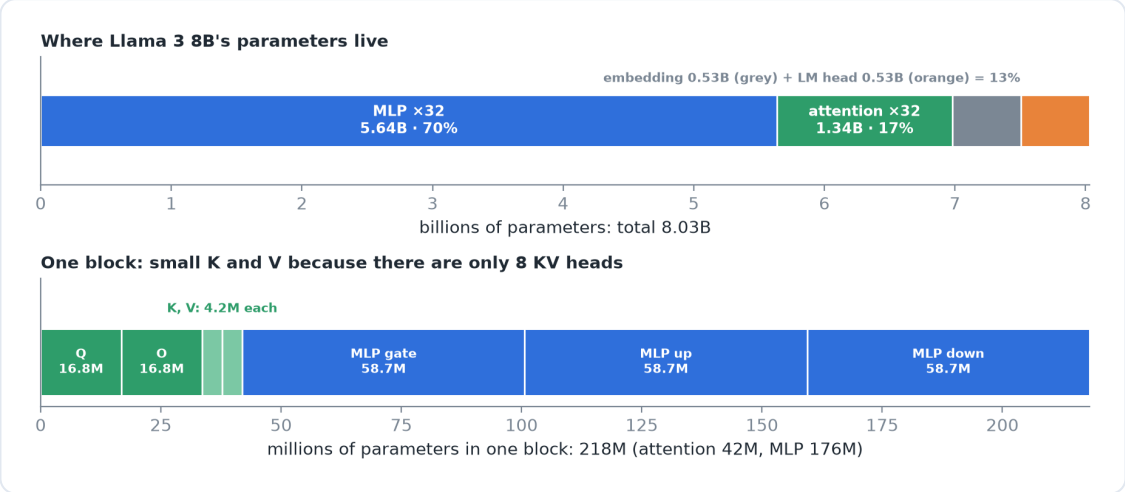
- Each token ID picks a **row of the embedding table**: a vector of **4,096 numbers**.
- The vectors pass through **32 identical blocks**. Each block has **attention** (tokens look at each other) and an **MLP** (each token processed on its own).

- Each part reads a **normalized copy** of the vector and **adds** its result back: that running vector is the **residual stream**.
- The **LM head** turns each final vector into a score for every vocabulary token (**128,256**).

> **KEY IDEA**
>
> That's the entire architecture. Everything an inference engine does is about running this stack faster.

2. Counting parameters from the config

Piece	Shape	Parameters
Embedding table	128,256 × 4,096	**~525M**
Attention Q, O (per block)	4,096 × 4,096 each	~17M each
Attention K, V (per block)	4,096 × 1,024 each (8 KV heads × 128)	~4M each
Attention total (per block)		**~42M**
MLP (per block)	3 matrices of 4,096 × 14,336	**~176M**
One block		**~218M** → × 32 ≈ **6.98B**
LM head (separate, not tied)	4,096 × 128,256	~525M
Total		**8.03B** (the "8B")



MLPs hold 70% of the weights, attention 17%, embedding + head 13%. Within a block, K and V are small because there are only 8 KV heads.

3. Parameters → bytes

Bytes = parameters × bytes per parameter (see 04): **16 GB** at 16-bit, **~8 GB** at 8-bit, **~4 GB** at 4-bit (plus a little for scaling factors). Decode reads **every byte every step**, so precision is both a **memory knob and a speed knob**.

> **INTUITION**
>
> The MLPs' 70% is where quantization saves most of the bytes, and most of the time.

4. Parameters → operations

Each weight is used once per token as **one multiply + one add**, so a token costs **~2 operations per parameter**, except the **embedding table**, which is a lookup (free). The LM head is a real matmul and one of the biggest matrices.

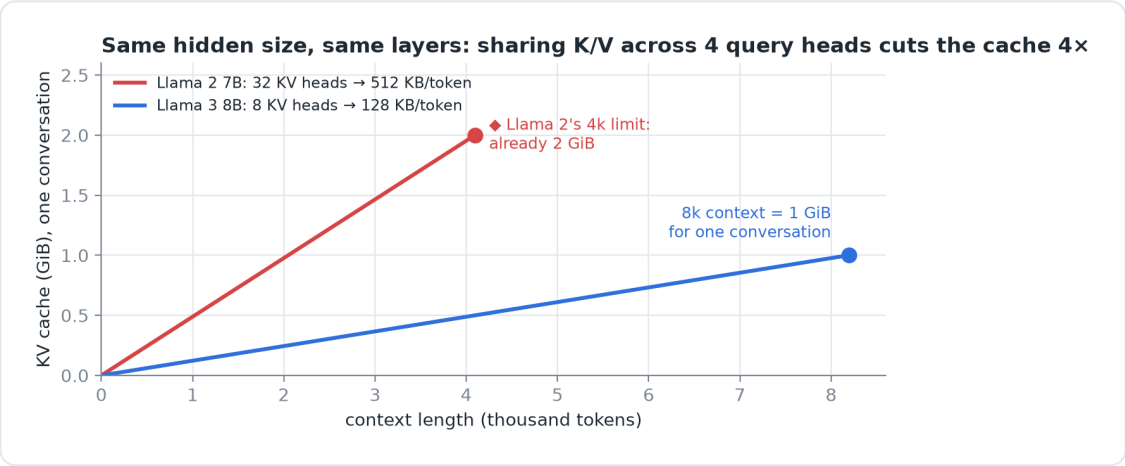
$$2 \times 7.5\text{B non}-\text{embedding params} \approx 15\text{B ops per token}$$

The same 15B whether the token is in the prompt or generated; what changes is **how they arrive**:

- **Prefill:** 500 prompt tokens together = **7.5 trillion ops** in one batch, keeping the math units busy.
- **Decode:** 15B ops against 16 GB of weight reads ≈ **1 op per byte**: the GPU mostly waits on memory (see 01, 04).

5. The KV cache: the line that grows

Every block keeps the **keys and values of every token it has seen**, so they're never recomputed. From the config: 8 KV heads × 128 × 2 (K, V) × 2 bytes = **4 KB per token per layer**, × 32 layers = **128 KB per token**. A full **8k context ≈ 1 GB**, for one conversation.

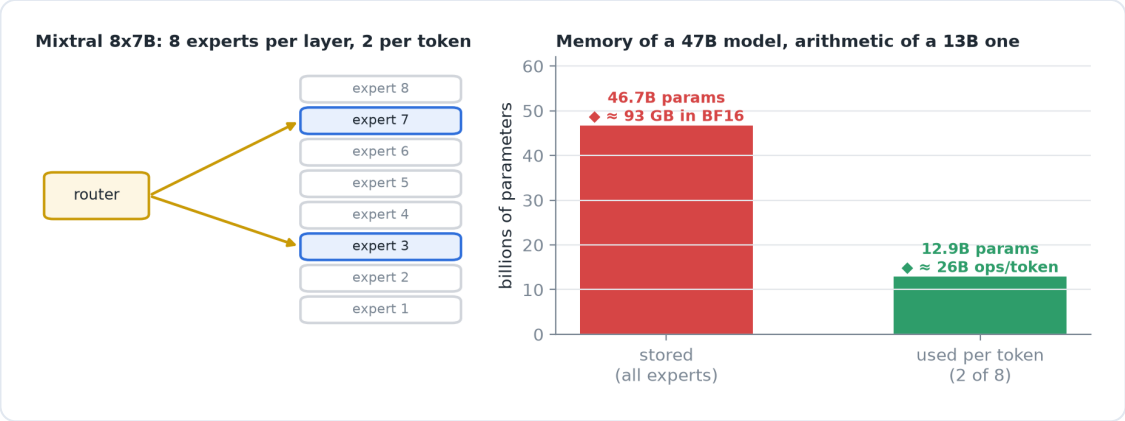


Llama 2 7B gave each of its 32 query heads its own K and V: 512 KB per token, 4× Llama 3 8B, with the same hidden size and layers.

> **KEY IDEA**
>
> Sharing keys and values across groups of query heads (GQA) is one of the cheapest wins in modern architectures.

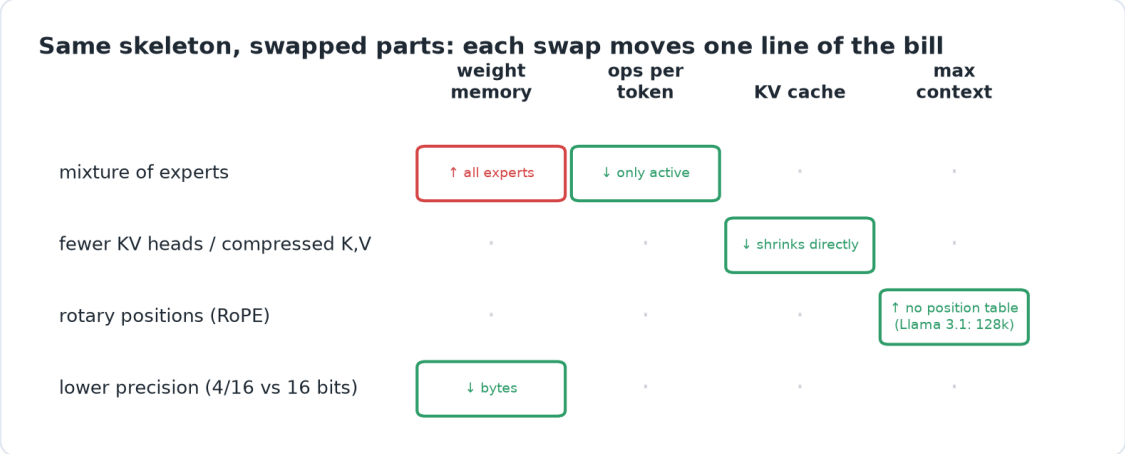
6. Variants on the same skeleton

- **Mixture of experts (MoE):** replace the one big MLP with many smaller **experts**; a **router** picks a few per token.



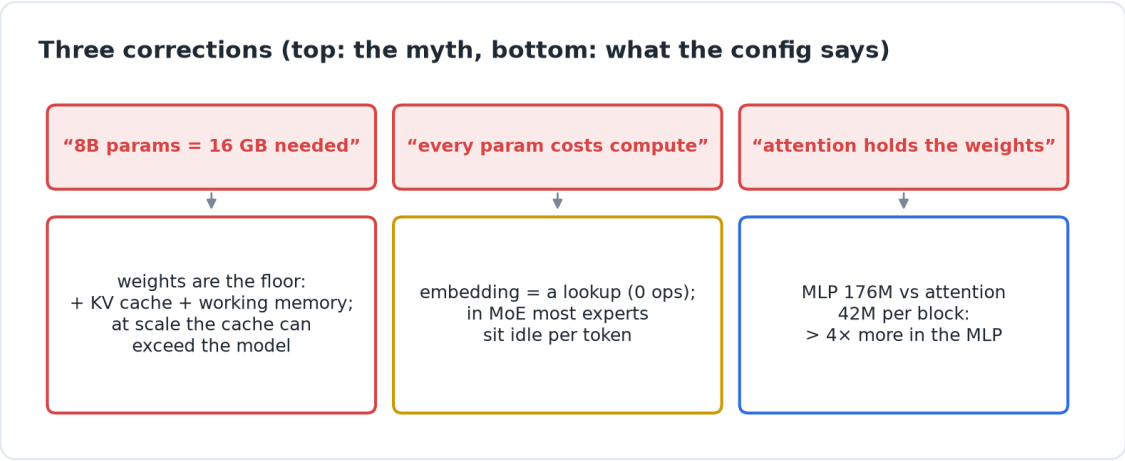
Mixtral 8x7B: 8 experts per layer, 2 per token. 46.7B parameters stored, 12.9B used per token: memory and compute come apart.

- **Attention variants:** fewer KV heads or compressed keys and values **shrink the cache line** directly.
- **Positions:** Llama rotates each query and key by a **position-dependent angle (RoPE)**. With no position table to outgrow, this is part of how **Llama 3.1 reached 128k tokens** of context.



Each swap shows up on exactly one line of the bill.

7. Three corrections



Parameter count isn't memory needed; not every parameter costs compute; the MLPs, not attention, hold most weights.

8. Recap

Lesson 07 on one card

SHAPE

embedding lookup → 32 blocks
(norm → attention, norm → MLP,
added to the residual stream) → head

PARAMETERS

8.03B from the config
MLP 70% · attention 17%
embedding + head 13%

OPS PER TOKEN

≈ 2 × non-embedding params
≈ 2 × 7.5B = 15B
same for prompt or generated

CACHE

128 KB per token (8 KV heads)
8k context = 1 GB
Llama 2 7B: 512 KB